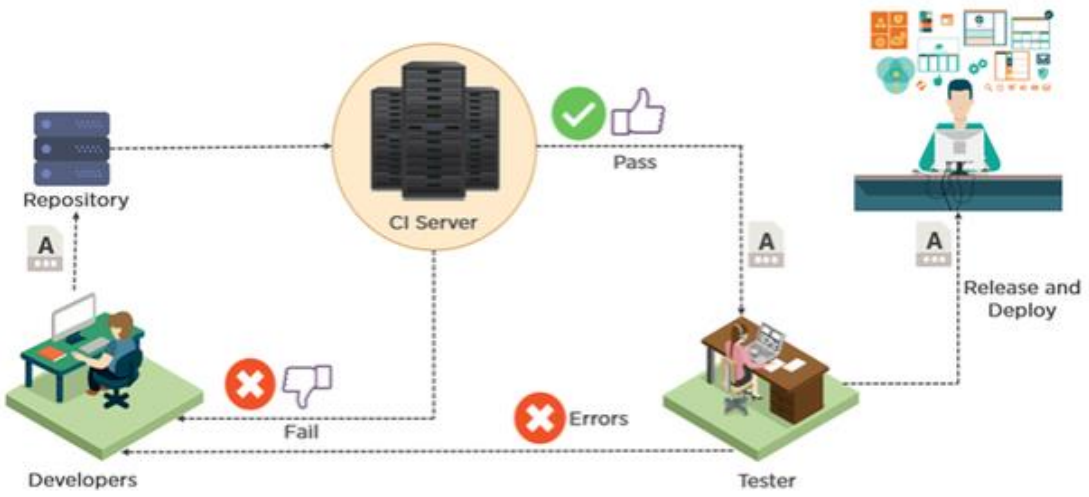
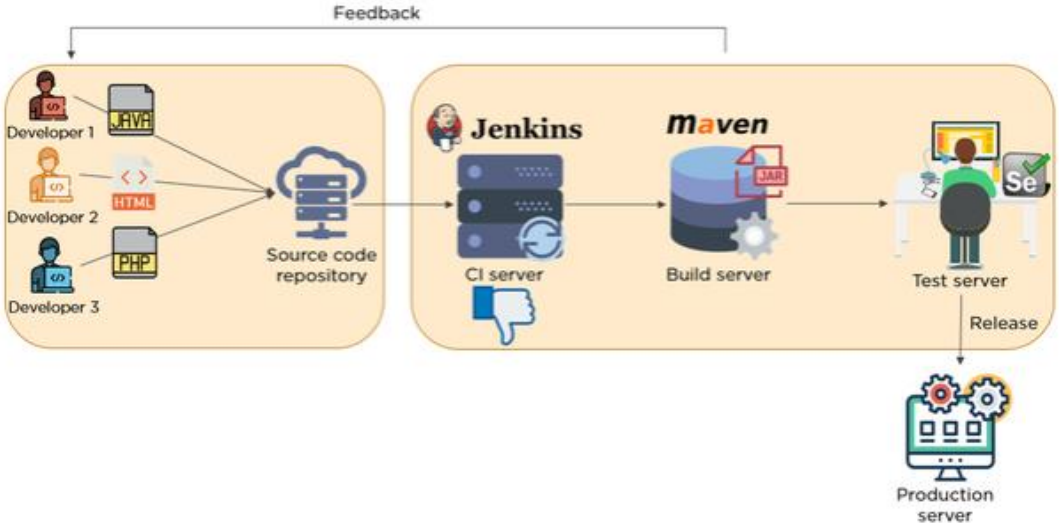
Jenkins

Jenkins is an open source continuous Integrtion server written in java that allows continuous development , build,test and deployments of codes.



Once application is ready our target is to deploy it in different environments, so that it will be available for n number of users.

We don’t deploy the source code, first we convert the source code into executable format (artifacts) this process is called build process.



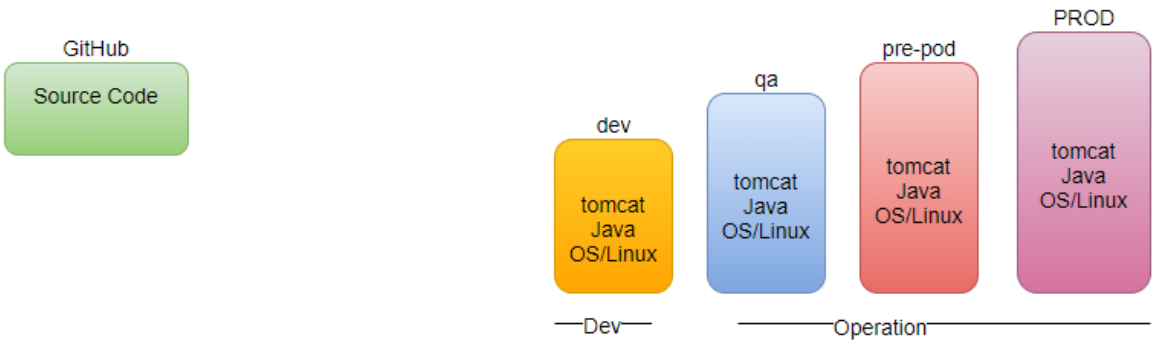
We don’t deploy directly in to the production before moving to production there are different environment where we test our application, just like covid19 vaccine testing phases.

Build Code: Is the process of converting source code to executable/artifacts.
java --.jar/.war/. ear
net -- .exe/.dll/msi
python – pybuilder

How to build code? Using the build tool
java -- **maven**/ant/Gradle
net -- vs/msbuild

Build code process: involves various stages
We can directly package sc to artifacts, but it is recommended to go through various stages

code review: static code analysis
unit test: developer level testing
code coverage: how much testing covered.
package: convert sc to executable



Maven:
What is Maven?
Maven is a **build tool** that is used to compile, validate codes and analysis the test-cases in the code.
Maven is also known as dependency management tool.
In maven there are three repositories.

1. local repository
2. center repository
3. remote repository

In first time it stores the file in **.m2** folder

How maven understand our code?

pom.xml (project object model) # developer writes the pom.xml

GAV

groupid: uniquely Identify your project.

artifactid: name of the package (.jar) without the version

versionid: version of the .jar

packaging tag

default: .jar

maven has three defined Lifecycles:

- default: build
- clean
- site:

Goal:

- Validate
- Compile
- Test
- Package
- Install

Maven creates a **folder target** # output of maven command

Each Lifecycle has their own phases

default: build

- validate
- compile
- test
- package
- integration-test
- verify
- install
- deploy

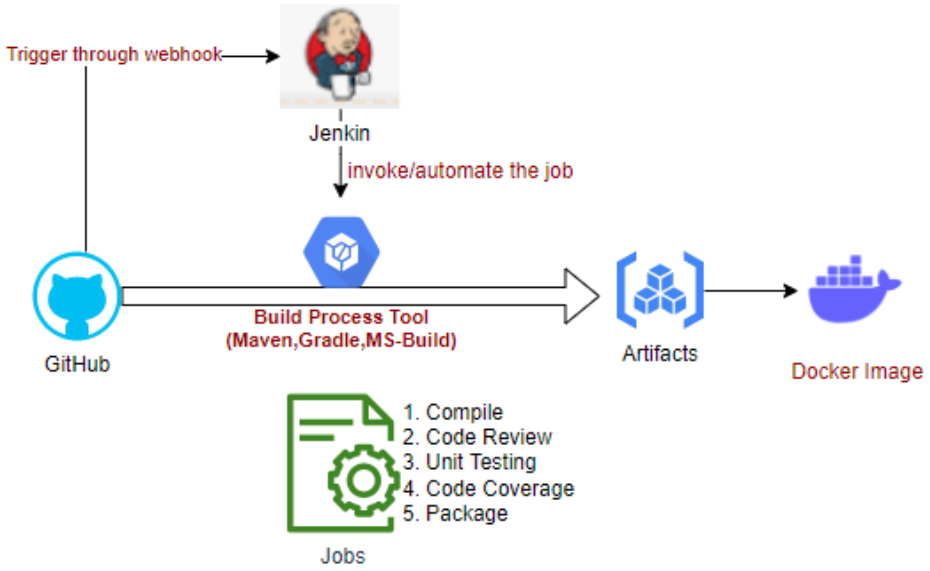
clean

- pre-clean
- clean
- post-clean

Maven is built and dependency management tool for Software projects.
It provides a DSL(.pom) to define, build, package and publish the project artifacts
To put it in a simple word, it's a tool to define how a project is built and how its dependencies are managed in order to build the project artifacts i.e., JAR, WAR or EAR of a Java project.

All the above steps performed by maven, then what is Jenkins? why we need Jenkins?
Jenkins: Build automation server, helps automating the build code process.

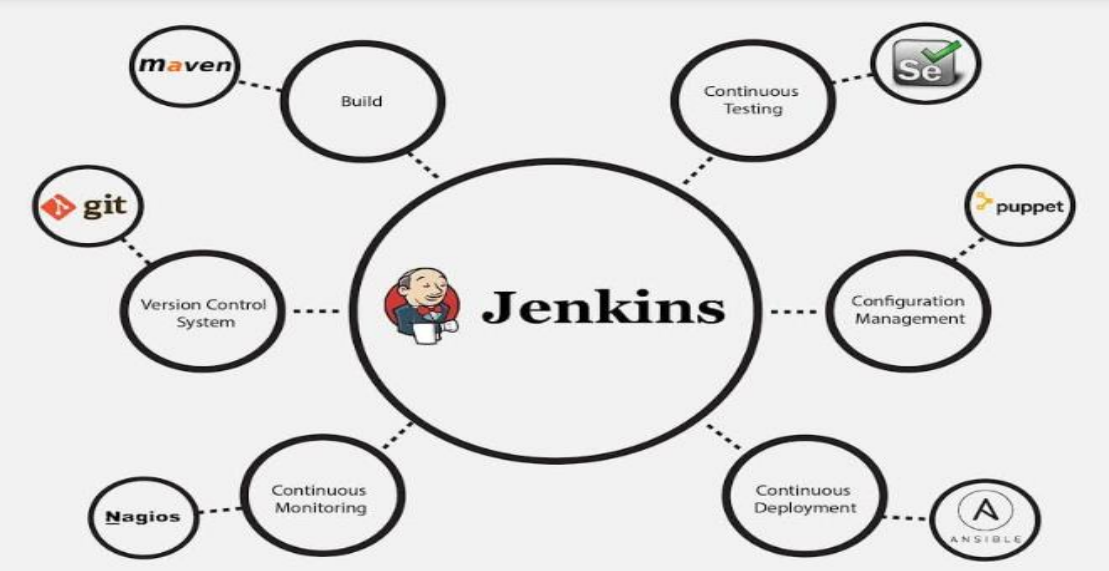
- We integrate Git with Jenkins
- We integrate java with Jenkins
- We integrate maven with Jenkins



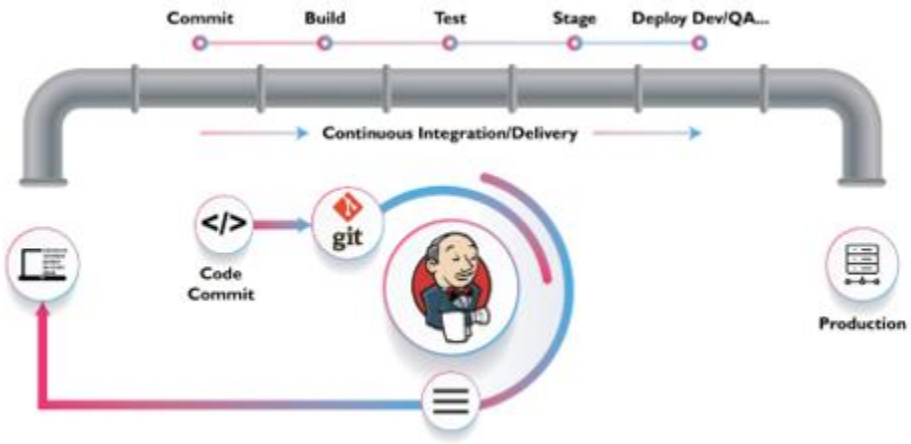
Job: Activities which we create.
Build: Execution of created jobs
We perform various stages of build code process continuously, that's why we call Jenkins as continuous integration server.

If we execute the various stages of build code process in a sequence manner that's called continuous integration pipeline (CI PIPELINE)

When we feel its **repetitive work go for automation**, where Jenkins comes in picture called **build automation server**, also called **continuous Integration server**. Jenkins is also called dummy tool it can't do anything its own, we have to give instruct to Jenkins hey go to git server take a latest code, hey Jenkins go and talk to build server make a build of latest code. This also called continuous Integration pipeline, it is a mediator collaborate with other tools also called orchestration tool this pipeline always executed whenever there is a commit in GitHub.
IaaC- Ansible, terraform, CloudFormation.
Jenkins's server repeats the tasks for you. It helps us to automate the entire build process.



Bigger Picture Role of Jenkins in DevOps?

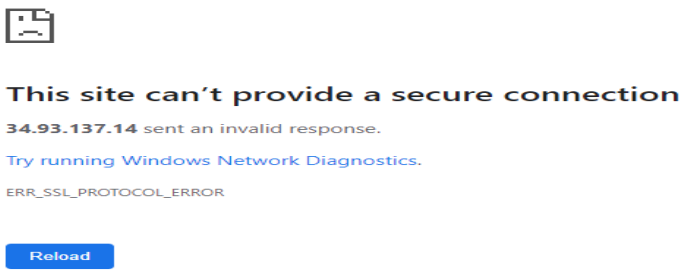


Installation:
Install java
Install Jenkins
Install Maven
JAVA_HOME: /usr/lib/jvm/java-8-openjdk-amd64
MAVEN_HOME: /opt/apache-maven-3.6.3
JENKINS_HOME: /var/lib/Jenkins
Jenkins is a tool integrate all tools together
Home->Manage Jenkins-> Global tool configuration

- **Java_Home**: /usr/lib/jvm/java-8-openjdk-amd64
- **Git:** which git
- **Maven_Home:** /opt/apache-maven-3.6.3

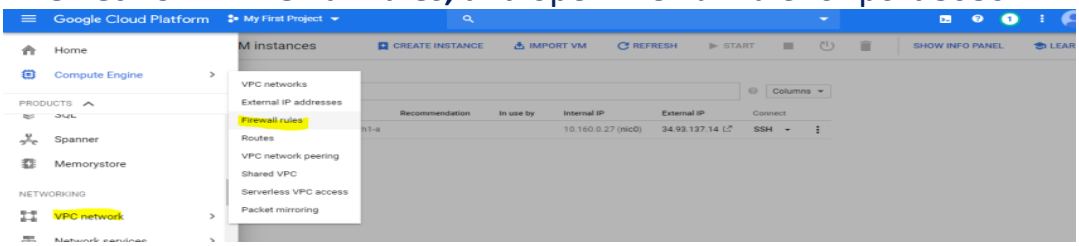
When we install a Jenkins, Jenkins run as a service/process.
service jenkins status

Why and how to open firewall ports in GCP?
Open the browser and run the Jenkins at port 8080, Open the browser and enter <Public Ip>:8080

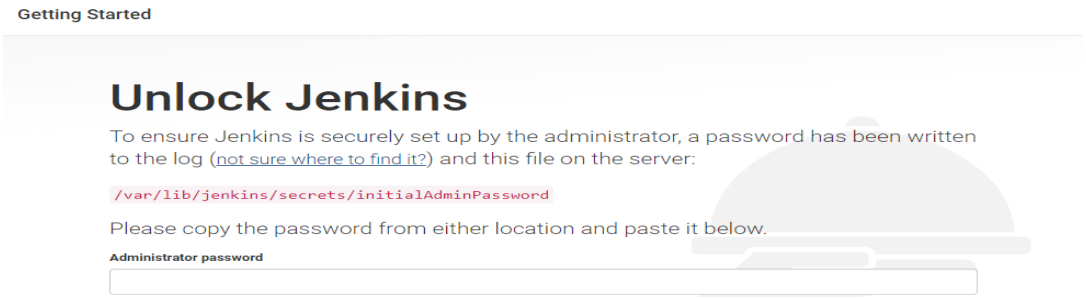


How?

VPC network-> firewall rules, and open firewall rule for port 8080



<Public Ip>:8080



cat /var/lib/jenkins/secrets/initialAdminPassword

Note: Replace the public Ip with private Ip

Jenkins is just a dummy tool; you must instruct to jenkins what to do.

How?

create a job (Item).

Without Jenkins, All the code is in our local, and we have to do the following steps manually.

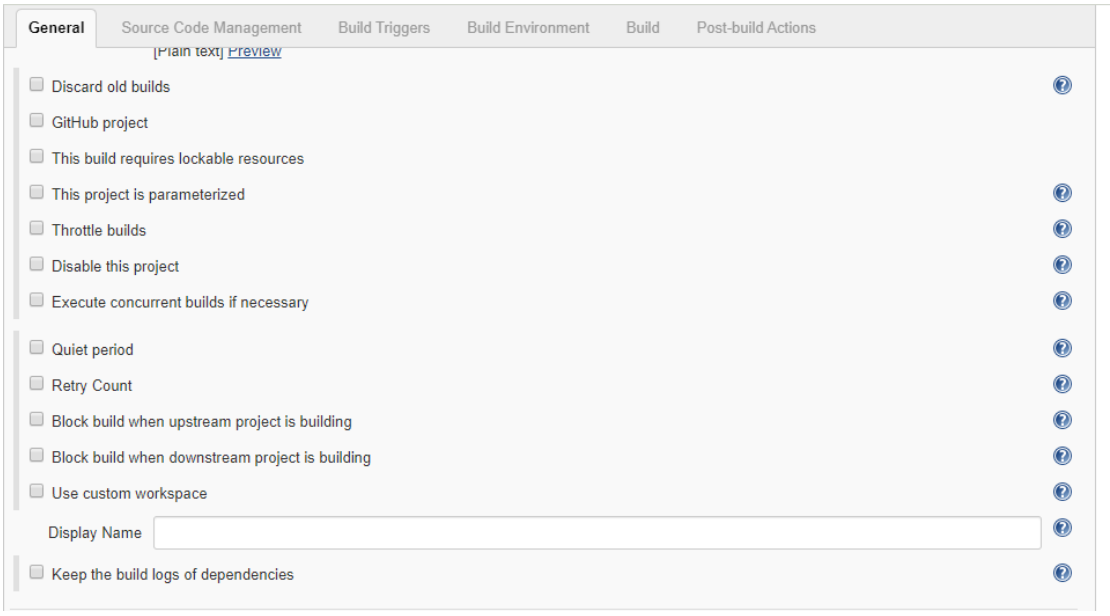
- Compile
- Code Review
- Unit Testing
- Code Coverage
- Package

```
root@instance-1:~/samplejavaapp#
root@instance-1:~/samplejavaapp# /opt/apache-maven-3.6.3/bin/mvn package
[INFO] Scanning for projects...
```

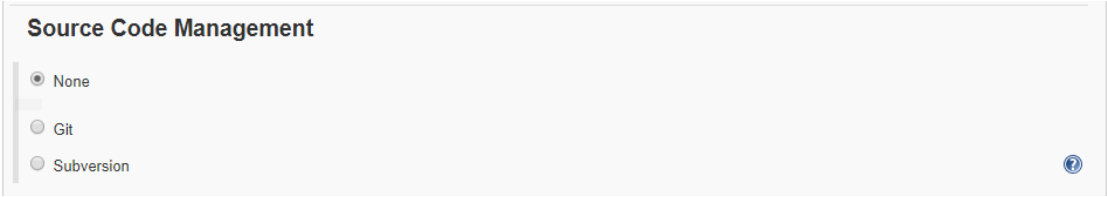
How we automate through Jenkins?

Jenkins Console

General: Description (Good Description What your job is going to do)



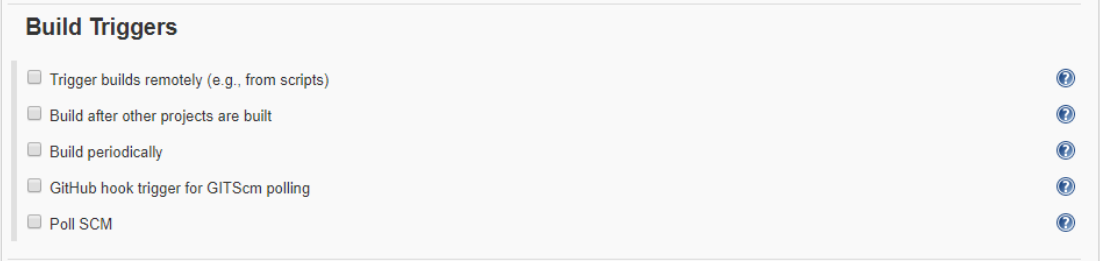
Source Code Management: job depends on some Repositories (Git Hub)



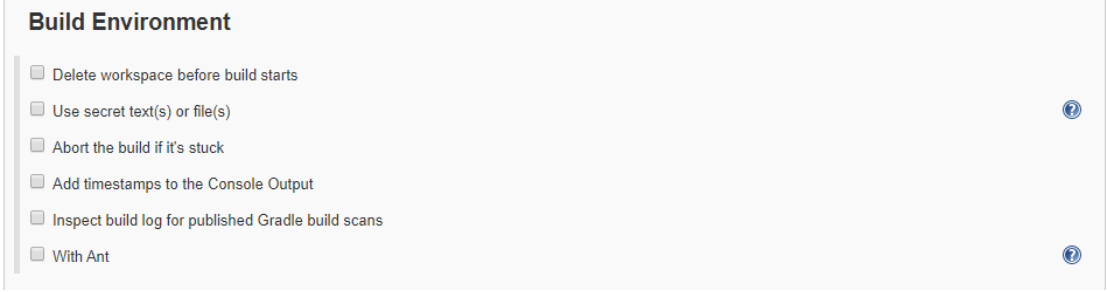
Build Triggers: You define when job will be triggered (certain time, event)

Remotely- token

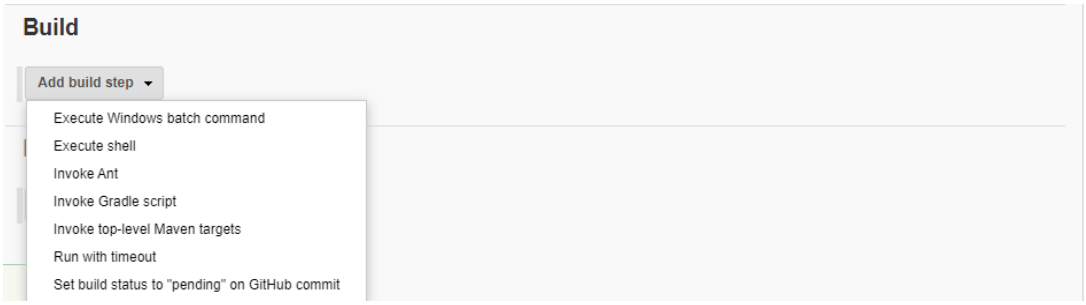
`<JENKINS_URL>`/job/1-compile/build?Token=`<TOKEN_NAME>`



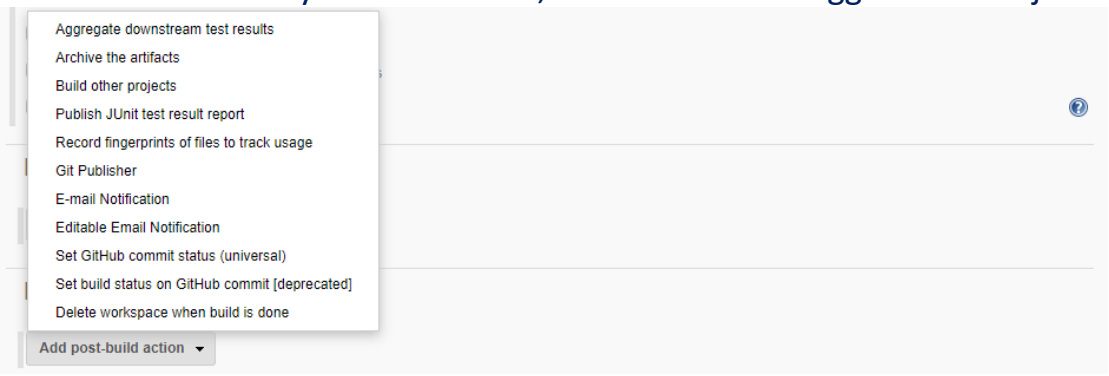
Build Environment: Where you define environment Variable



Build: What has to be done by this job



Post-build action: Once your build done, what after that trigger another job?

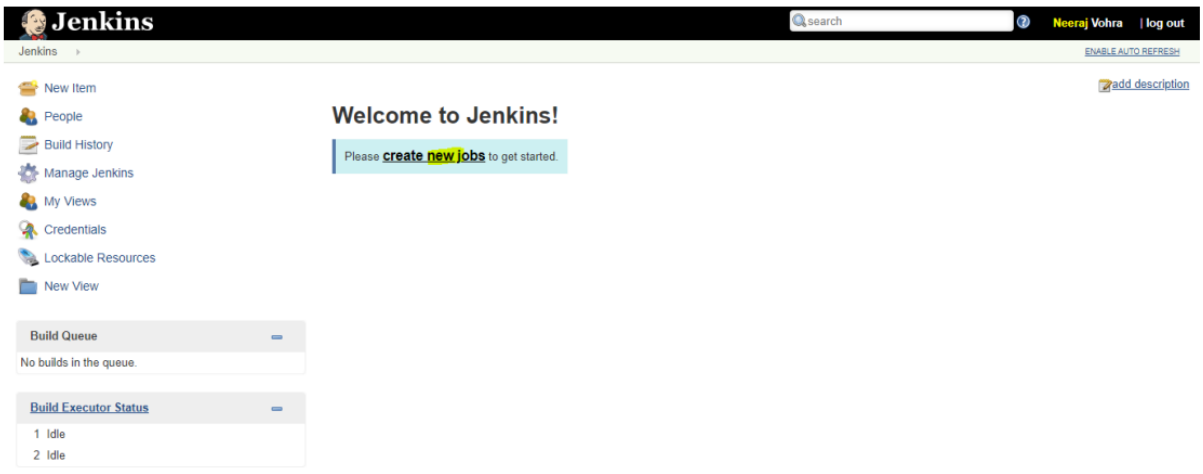


How to create a new job in Jenkins?

Project Type: Freestyle/Pipeline/…
General:
Source Code Management:
Build Trigger:
Build Environment:
Build:
Post Build Actions:

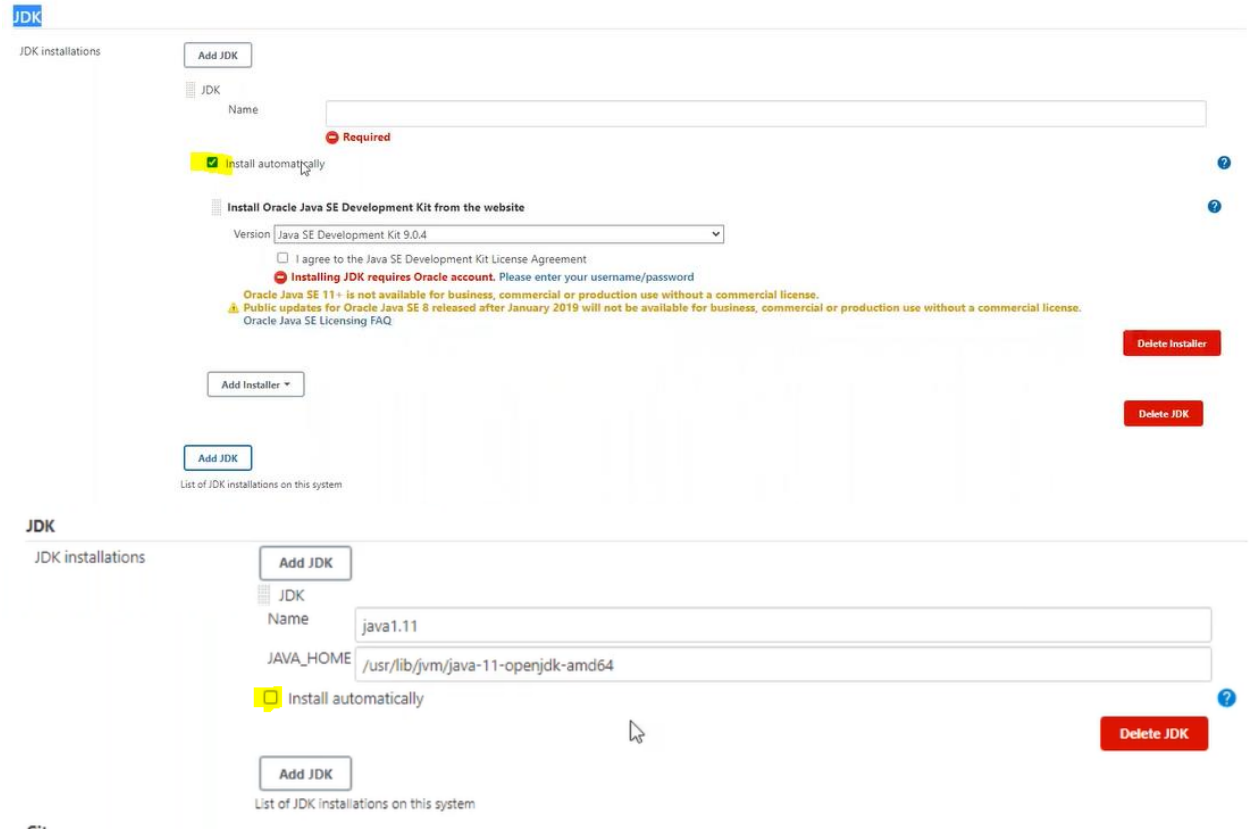
Example: Test Job

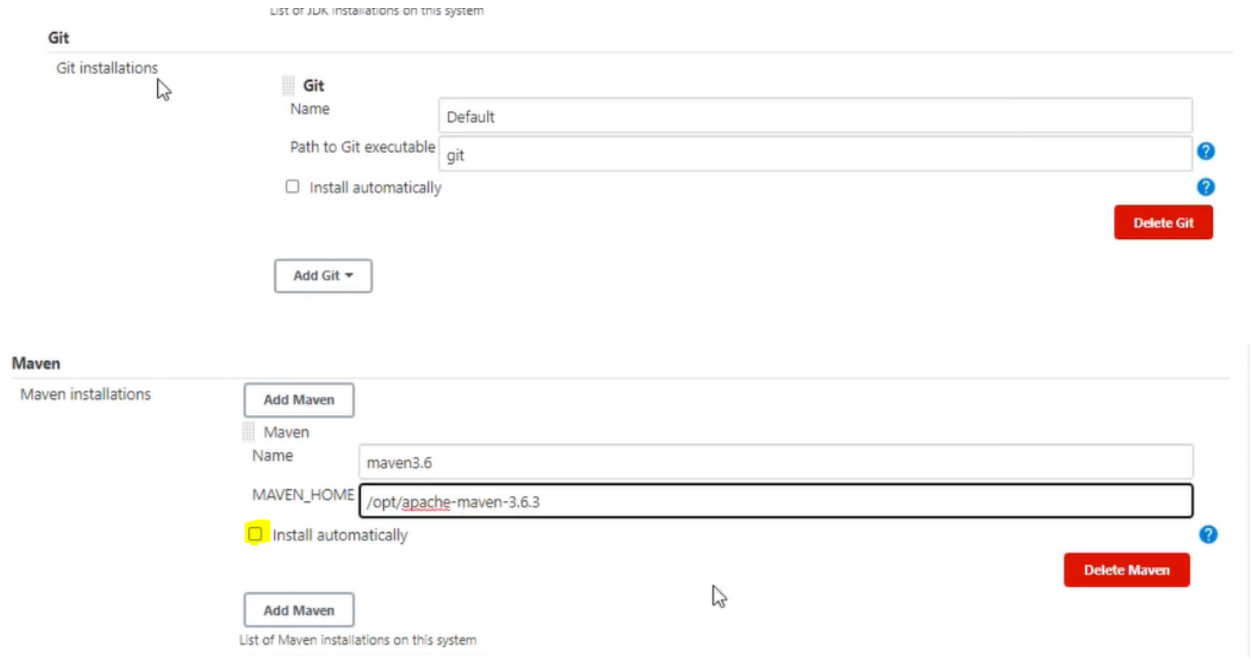
Home Page



Project Type: Free Style
General: Very Simple Job
Source Code Management:
Build Trigger:
Build Environment:
Build: Execute Shell (echo “hello world”)
Post Build Actions:
Home Page->Test Job-> Build Now
Jenkins never override previous build, always creates new build.
Default Job created in /var/lib/Jenkins/workspace/<Name of Job>

System Configuration:
Manage Jenkins->Global Tool Configuration:





What is plugin?

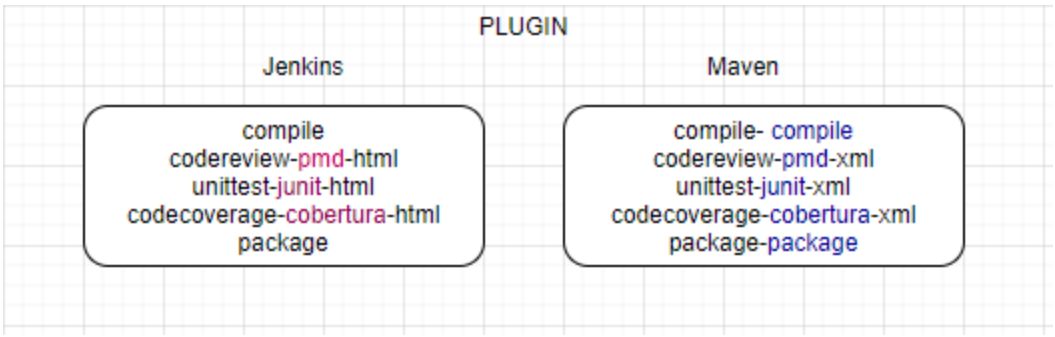
Enhance the functionality of basic S/w.

mobile phone comes with basic functionalities, to add more features we have to go to play store and add additional features, similarly jenkins comes with basic functionalities, just like maven comes with basic feature with compile, now we want code review but **code review functionality not a default feature of maven**, it needs some plugin to extend the functionality of the tools, need **pmd (program mistake detector) plug-in** to enhance the functionality of maven to do the code review, will take from the pom.xml.

Similarly, **code coverage** is done by another plugin cobertura.

Similar Jenkins comes with various functionalities and can enhance its functionalities by adding corresponding plug-in. various integration can be done with Jenkins to perform various tasks.

Jenkins is very popular for its plugin itself; plugin helps to Jenkins to enhance it up to any level.

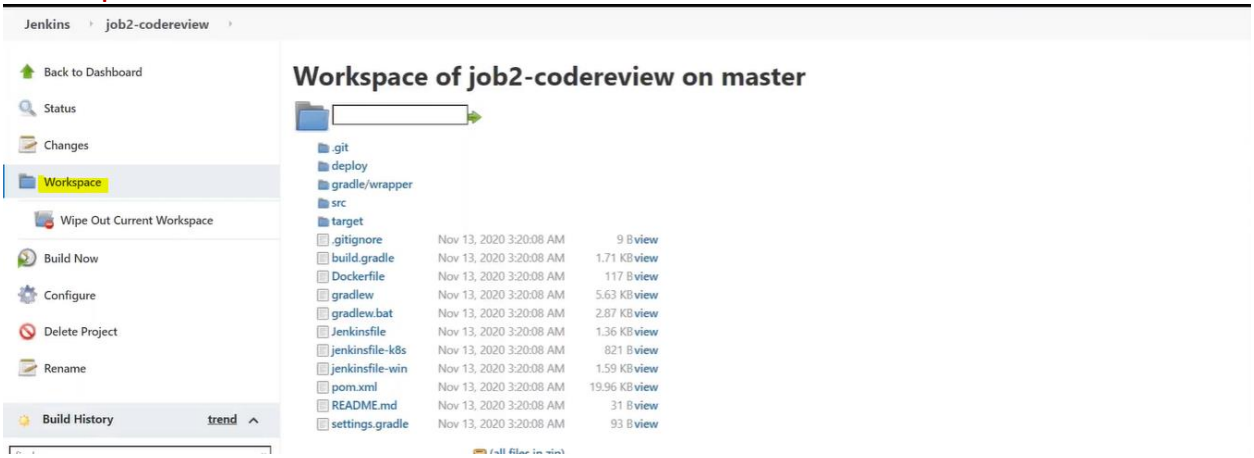


UI shown, based on added plugins, after adding Plugin some more block will be visible e.g., after adding audit trial Plug In, Logger block will be visible.
Jenkins enhanced its functionality with the help of plug-in, Jenkin support approx. (1500+) plug-in. **(plugins.jenkins.io)**
Manage Jenkins-> Manage Plugins
Global Variables
/env-vars.html
Use Case: Understand how manually application working?
Compile->Test->Review->Package->Deploy
steps to clone and compiling samplejavaapp manually
mkdir tmpapplication
cd tmpapplication
git clone https://github.com/NeerajVohra/samplejavaapp.git
mvn compile
mvn test
Continuous Integration (Using Jenkins)
Compile->Test->Review->Package (CI)
step1: Create a new job ApplicationCompile
Project Type: Free Style
General: Pull the Source Code from GitHub and Compile
Source Code Management: Git URL
https://github.com/NeerajVohra/samplejavaapp.git
Git understands, it has to clone the remote repository.
/var/lib/jenkins/workspace/<job name> (keeps output of job in /target folder)
Build Trigger:
Build Environment:
Build: **invoke top-level maven targets** (option will available once we install maven in Jenkins (global tool configuration). Uninstall maven from system (apt-get remove --purge maven)
Goal-> compile

Post Build Actions: NA

/var/lib/jenkins/workspace/<job>/target (contains output generated by job)

Workspace can be viewed from console.



step2: Create a new job ApplicationReview.

Project Type: Free Style

General: Pull the Source Code from GitHub and Review pmd (program mistake detector)

Source Code Management: Git URL

https://github.com/NeerajVohra/samplejavaapp.git

Build Trigger: Build after other projects are built

Build Environment:

Build: invoke top-level maven targets

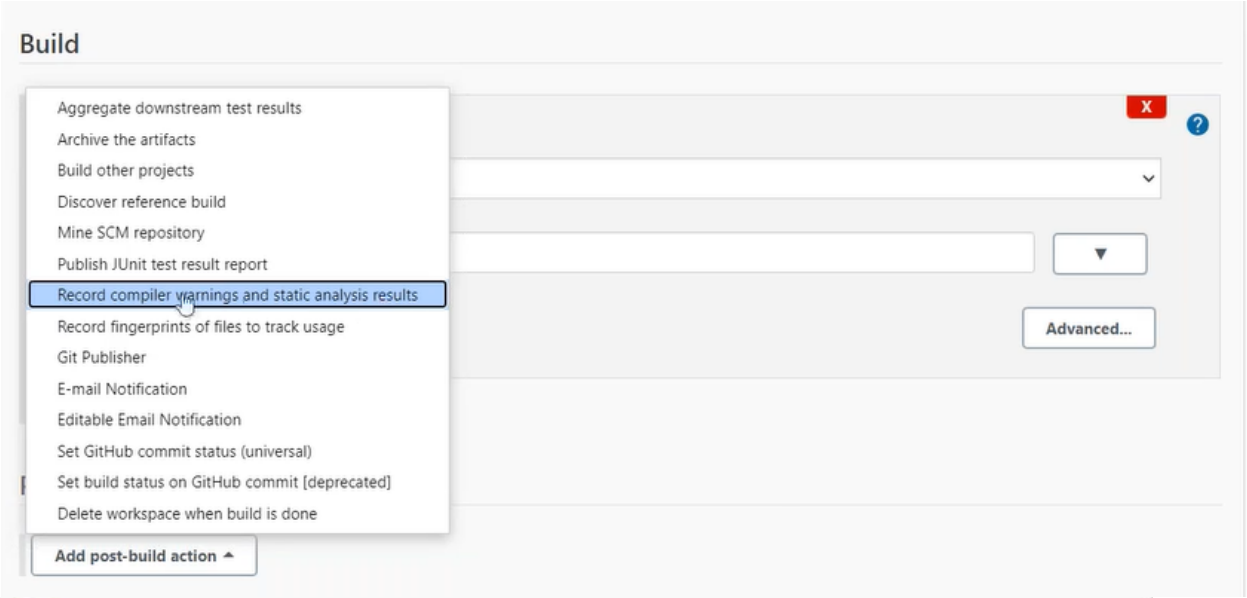
Goal-> pmd:pmd -P metrics pmd:pmd

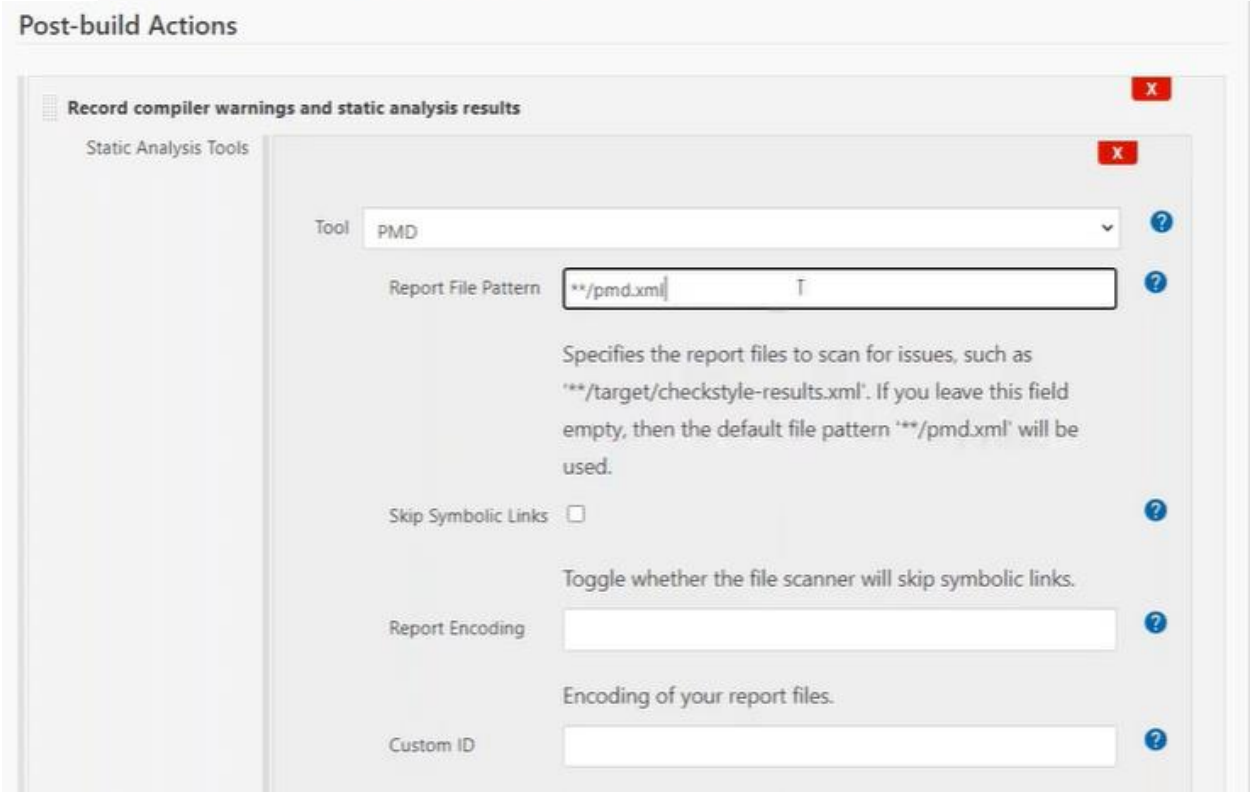
Maven generates the report in xml format, we can instruct Jenkins, **hey! Jenkins, please convert the generated file in UI format**. (add the Jenkins Plugin for the same).

Post Build Actions: Record Compiler Warnings and static analysis Result (If option not available then first install the **Warnings Next Generation Plugin.**

Click on Add post-build action (select Record compiler warnings and static analysis results)

Post Build Action:





**/pmd.xml // ** means /target folder for Jenkins /var/lib/jenkins/workspace/<job>/target (contains output generated by job)

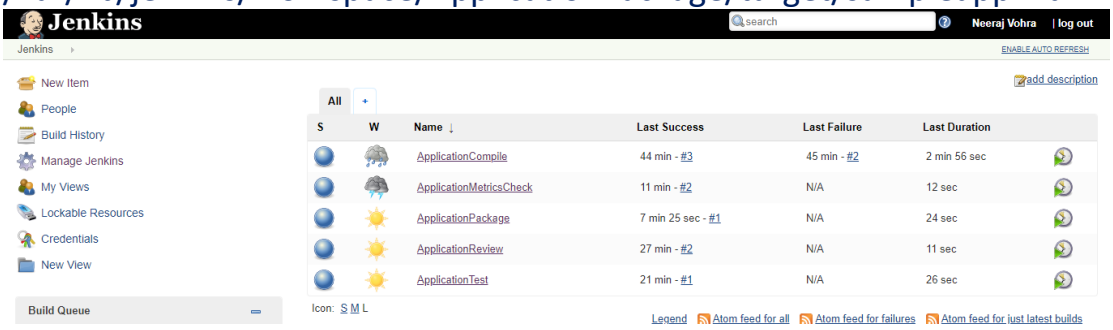
step3: Create a new job ApplicationTest.
Project Type: Free Style
General: Pull the Source Code from GitHub and Unit Test

Source Code Management: Git URL
https://github.com/NeerajVohra/samplejavaapp.git
Build Trigger:
Build Environment:
Build: invoke top-level maven targets
Goal-> test
Post Build Actions: publish Junit test result report
Test report XMLs: target/surefire-reports/*.xml
/var/lib/jenkins/workspace/<job>/target /surefire reports (contains output generated by job

step4: Create a new job ApplicationMetricsCheck.
Project Type: Free Style
General: Pull the Source Code from GitHub and code coverage
Source Code Management: Git URL
https://github.com/NeerajVohra/samplejavaapp.git
Build Trigger:
Build Environment:
Build: invoke top-level maven targets
Goal-> cobertura: cobertura -Dcobertura.report.format=xml
Post Build Actions: publish Cobertura Coverage Report (If option not available then first install the Cobertura plugin) from home->manage jenkins-> manage plugin
Cobertura xml report pattern target/site/cobertura/coverage.xml
/var/lib/jenkins/workspace/<job>/target (contains output generated by job)

step5: Create a new job ApplicationPackage.
Project Type: Free Style
General: Pull the Source Code from GitHub and create .war
Source Code Management: Git URL
https://github.com/NeerajVohra/samplejavaapp.git
Build Trigger:
Build Environment:
Build: invoke top-level maven targets
Goal-> package
Post Build Actions:
/var/lib/jenkins/workspace/<job>/target (contains output generated by job)

/var/lib/jenkins/workspace/ApplicationPackage/target/sampleapp.war



All jobs running properly independently.

- ApplicationCompile
- ApplicationReview
- ApplicationTest
- ApplicationMetricsCheck
- ApplicationPackage

How to integrate all Independent Jobs?

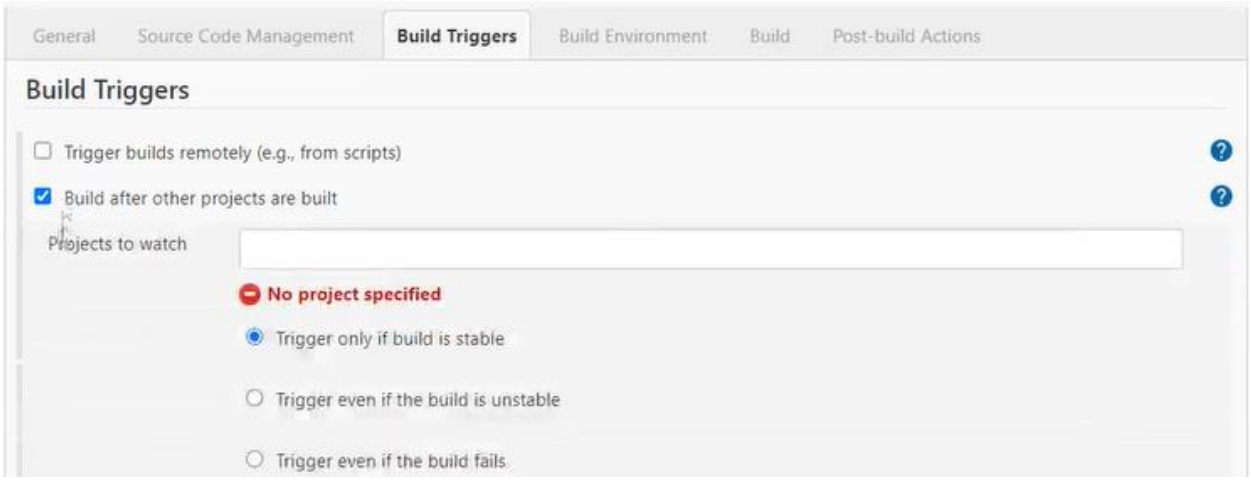
Update the configuration of each project->post build actions of each job

Post Build Actions->build other projects

(or)

Update the configuration of each project-> build trigger actions of each job

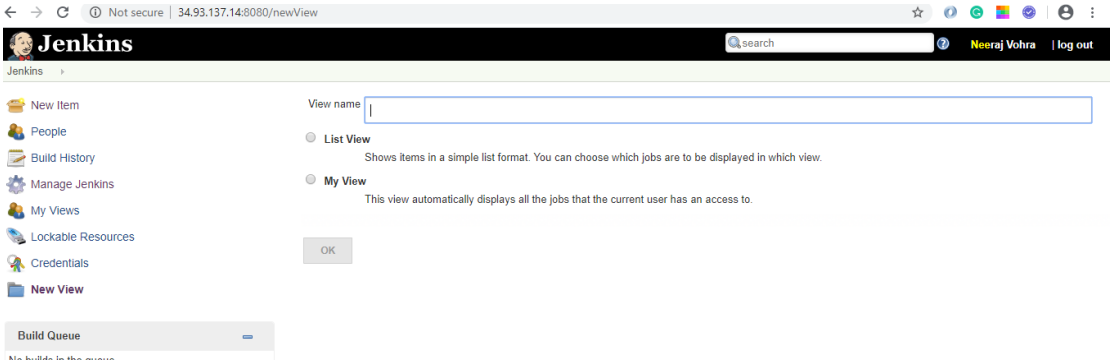
Build Triggers->build after other projects are built



How wrap all Jobs around pipeline?

Click on + sign

No View for Pipeline



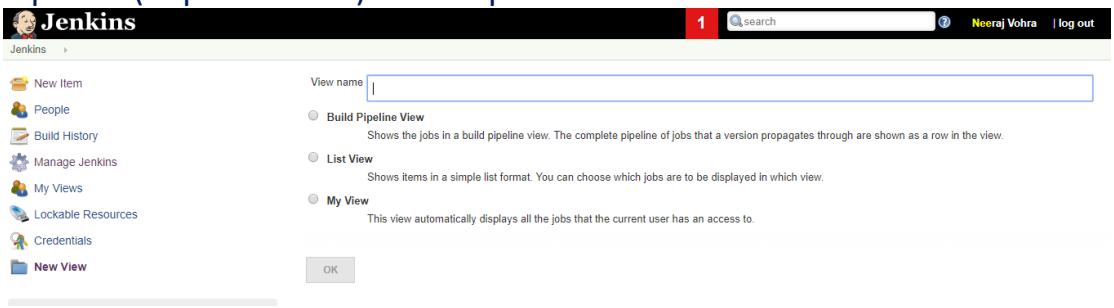
How to visualize dependencies which job depends on which job? Dependencies between jobs are configured as upstream and downstream configure in Jenkins? job3 is dependent on job2, job2 is upstream (previous job) for job3, and Job4 is downstream (next job) for job3.

Pipeline (Automate CI process)

Home->Manage Jenkins-> Manage Plugins (add **build pipeline** plugin)

Click on + sign

Pipeline (dependencies) Build Pipeline View visible

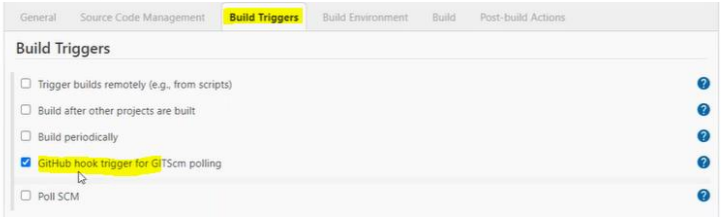


step1: view name: sampleapp_CI

step2: select pipeline tab-> Pipeline Flow (Upstream (previous job)/downstream (next job) config)

Select the initial job (ApplicationCompile)

How to trigger the first job of the pipeline?



Poll SCM

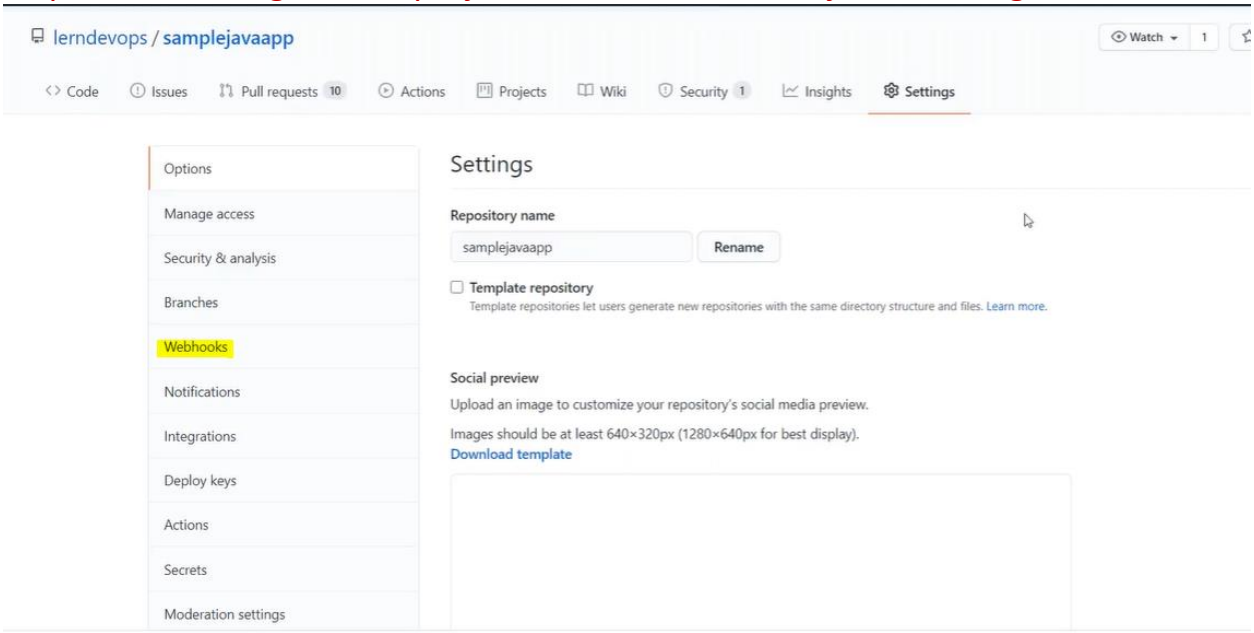
Jenkins's server poll the GitRepo, as per schedule. If there is any change in repository then Jenkins will execute the Job.

Build trigger Webhook?

Git Itself invokes the job, in case of any push.

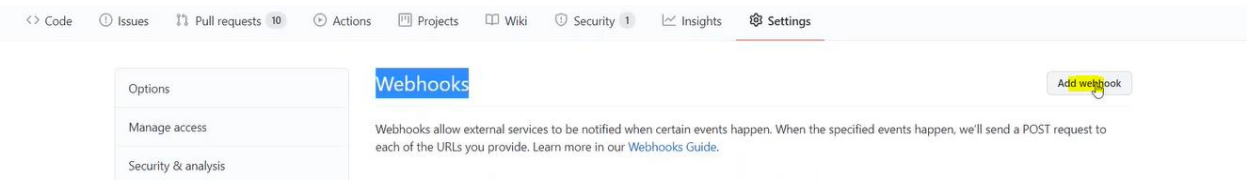
step1: select the option GitHub hook trigger for GitSCM pooling

step2: In GitHub, go to the project as owner of the Project-> setting

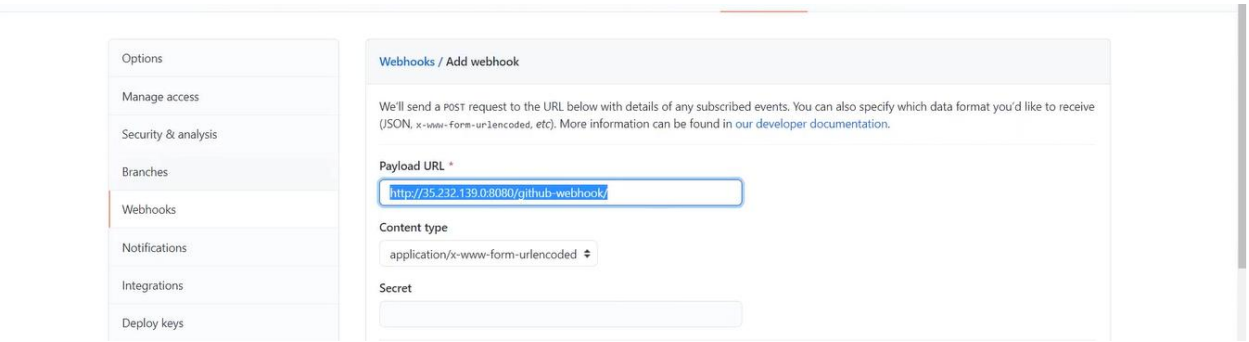


step3: select the webhooks

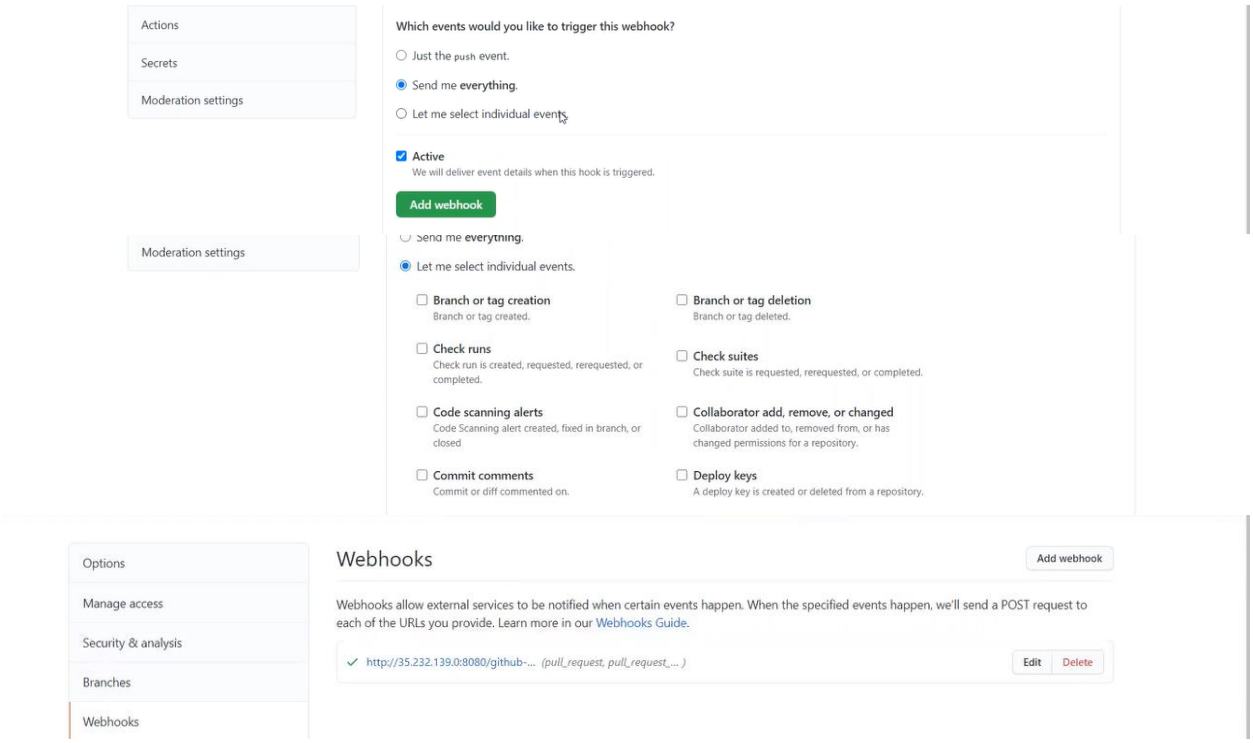
step4: click on add webhook

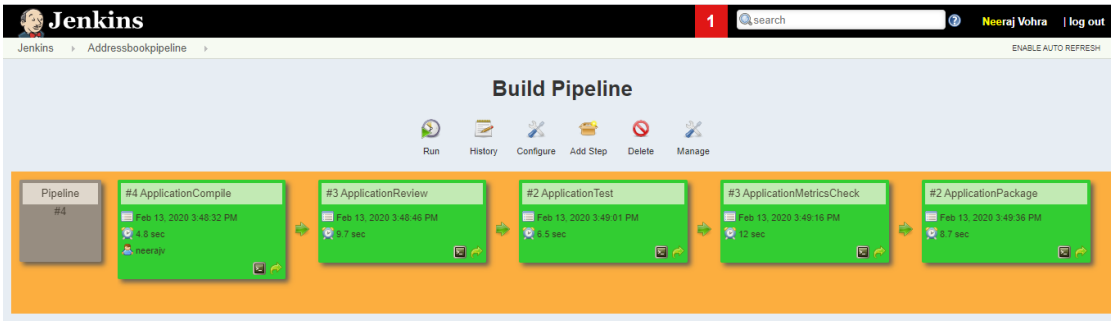


step5: enter the payload URL: 'Jenkins URL: port/github-webhook/



step6: select the event when webhook invoked

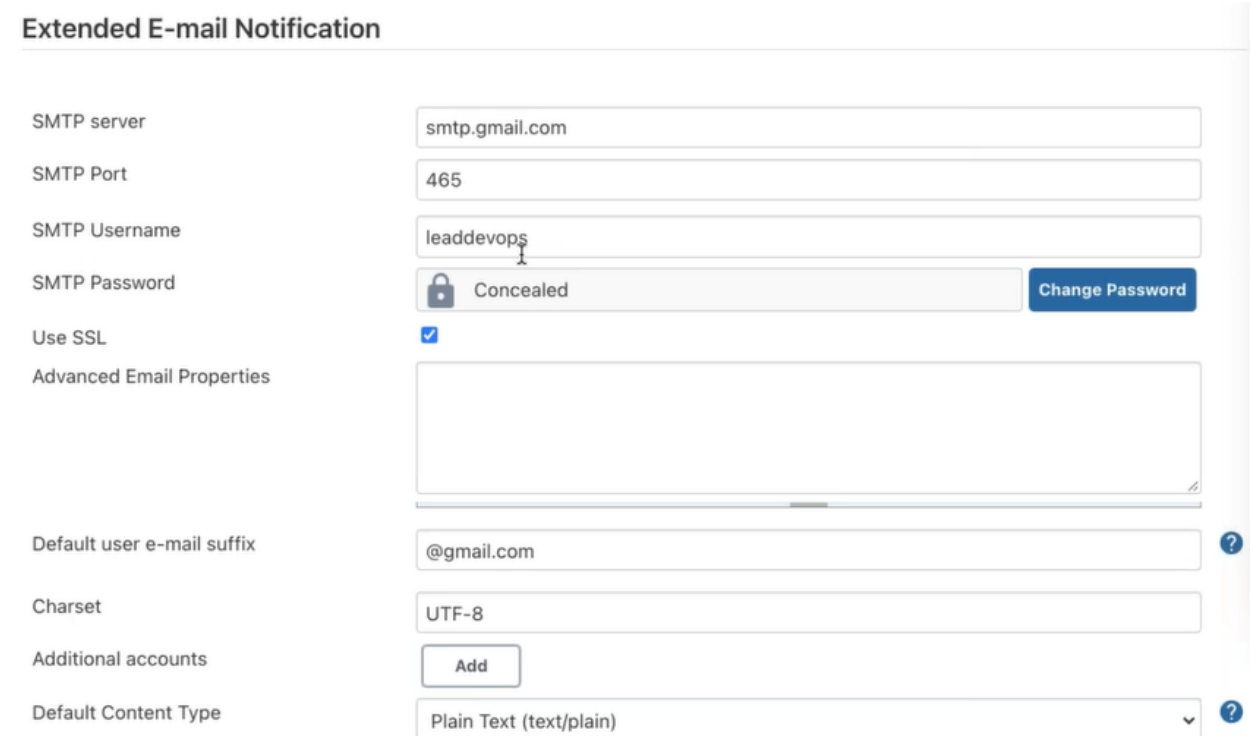




Email Configuration

step1: Home-> manage Jenkins-> Configure Systems (servers we don't install in Jenkins running somewhere else just configuring in Jenkins through configure system).

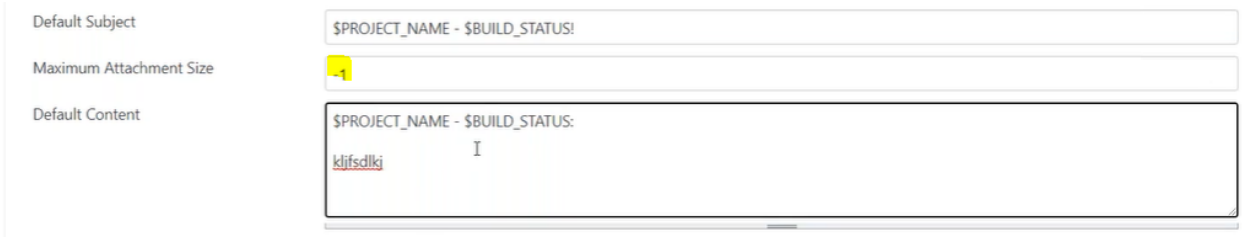
SMTP servers maintained by system team can get configuration information from them.



5 parameters are mandatory (SMTP server, SMTP port, SMTP username, SMTP password, check the Use SSL)

maximum attached size -1 (unlimited)

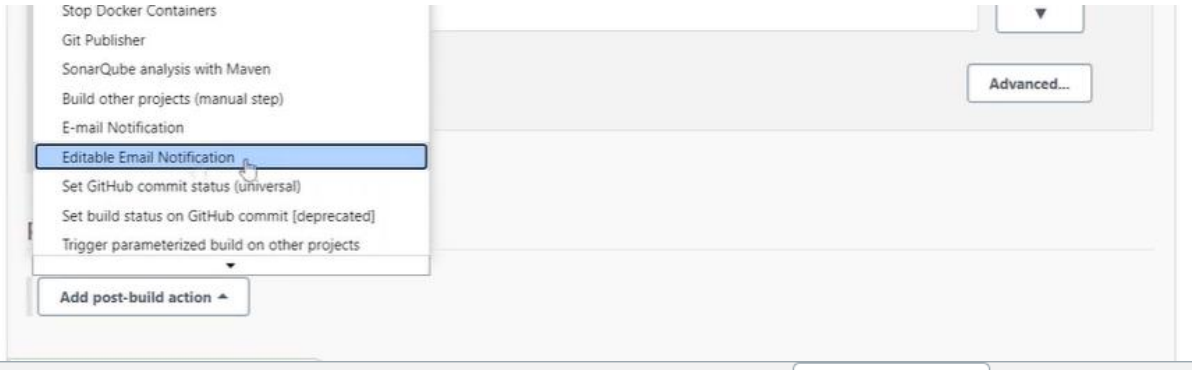
default Jenkins's variables

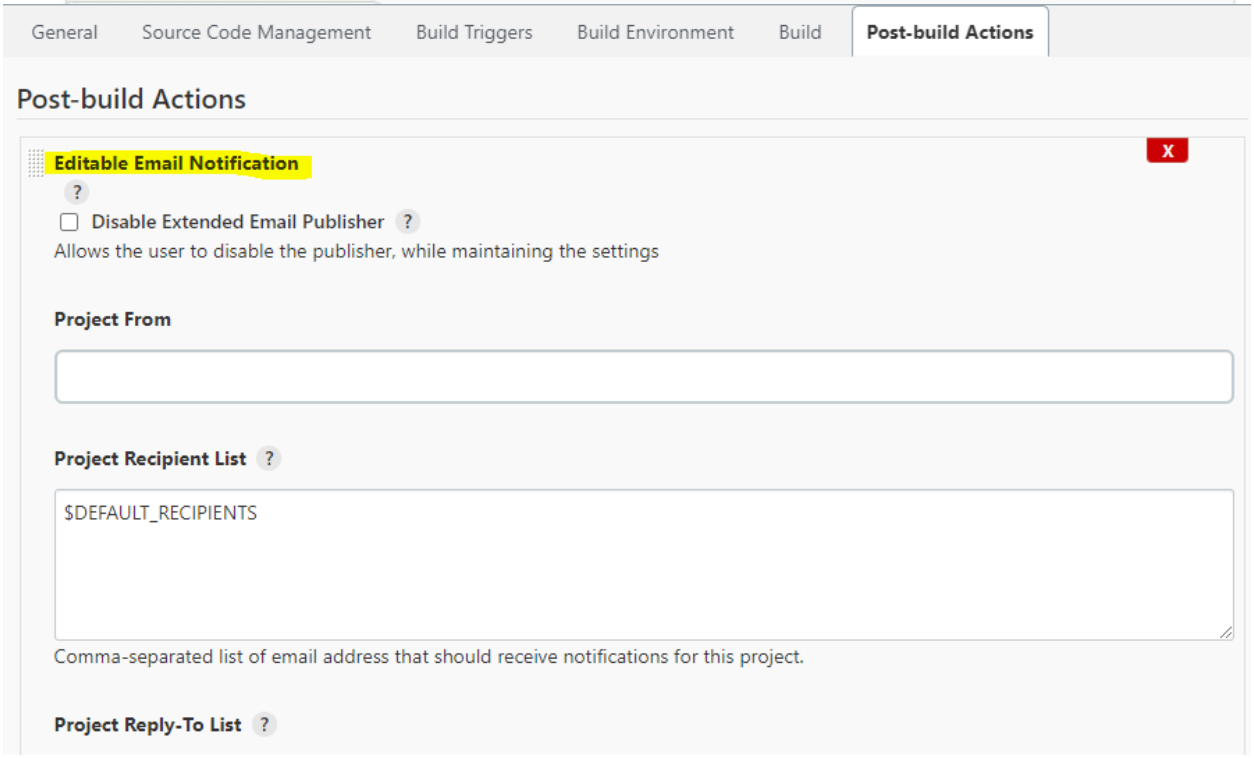


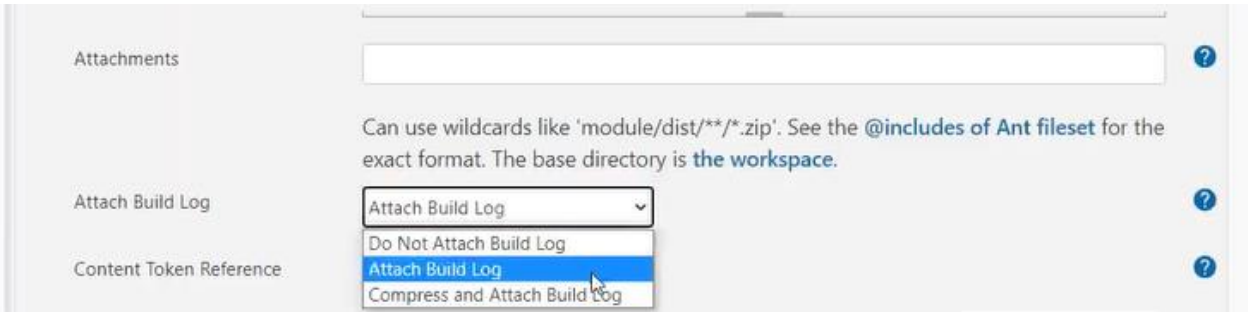
step3: Click on Default Triggers

step4: After doing the settings at configuration level, go to Job. (you want to send the mail)

Post build action







step5: In google account disable the security option

How do I remove Google security settings?

- Turn off 2-Step Verification
- Open your Google Account.
- In the "Security" section, select 2-Step Verification. You might need to sign in.
- Select Turn off.
- A pop-up window will appear to confirm that you want to turn off 2-Step Verification. Select Turn off.

Where is security option in Google Account?

- Click on the blue My Account button to access your account's settings.

Jenkins's security configuration?

- Authentication
- Authorization

Home-> manage Jenkins-> configure Global Security.
Home-> manage Jenkins-> manage user (By default Jenkins own user database)

Authentications:

- Jenkins own user database
- LDAP
- Unix user/group database

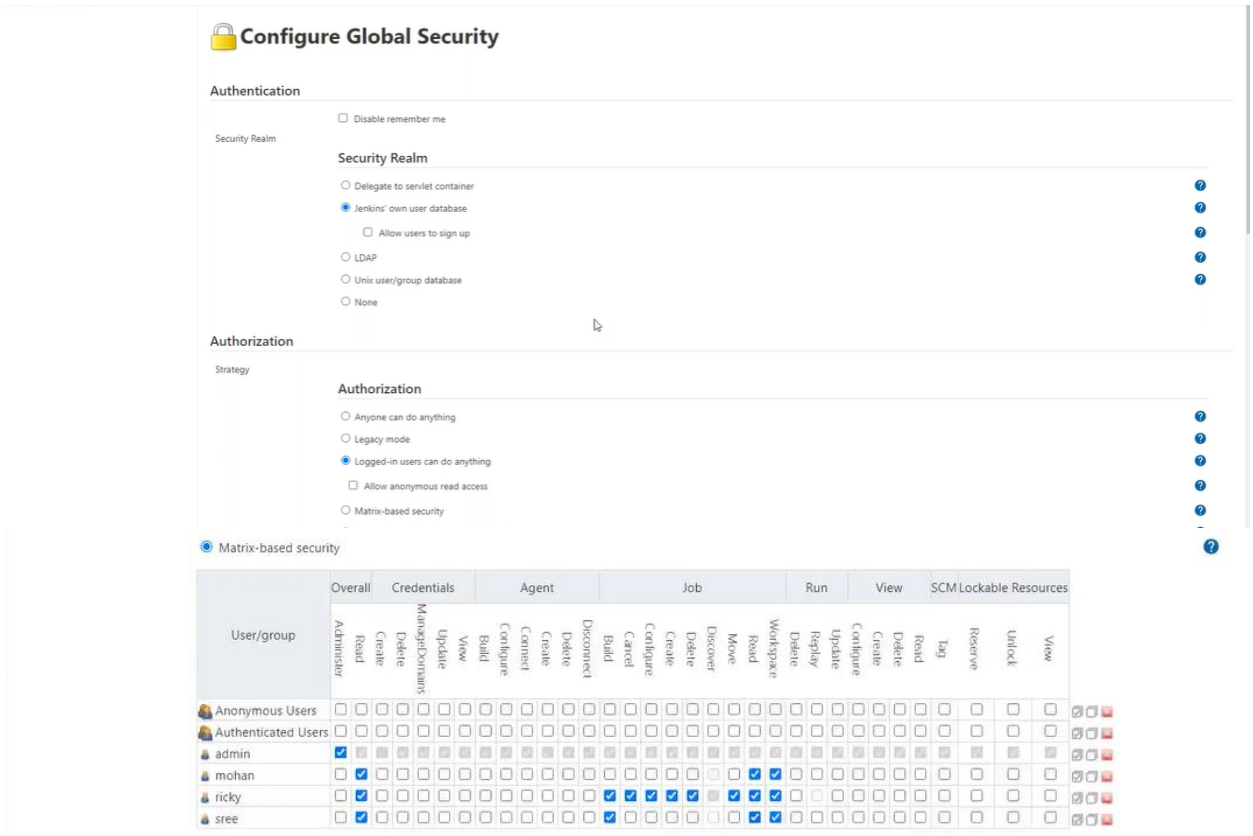
Jenkins stores everything in its own database by default.
/var/lib/jenkins/users

step1: create user.
manage jenkins->manage users-> create user.
/var/lib/jenkins/users
ls -l
more users.xml

step2: by default, created user have admin privileges.

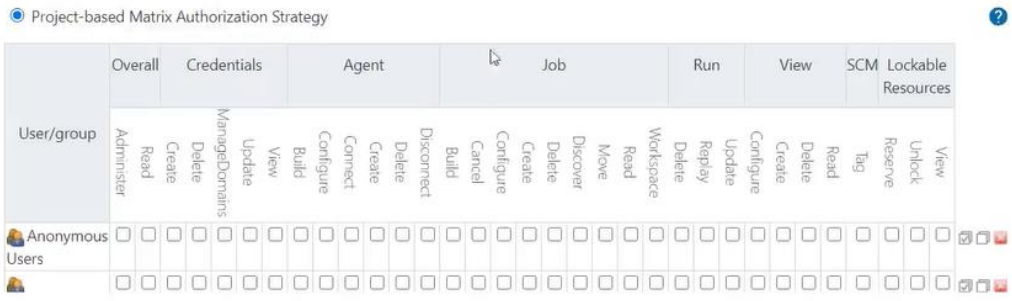
step3: restrict the access to newly created user.

- manage jenkins-> configure global security (authorization).
- Anyone can do anything.
- Legacy mode.
- Logged-in users can do anything.
- Matrix-based access
- Project-based matrix authorization strategy



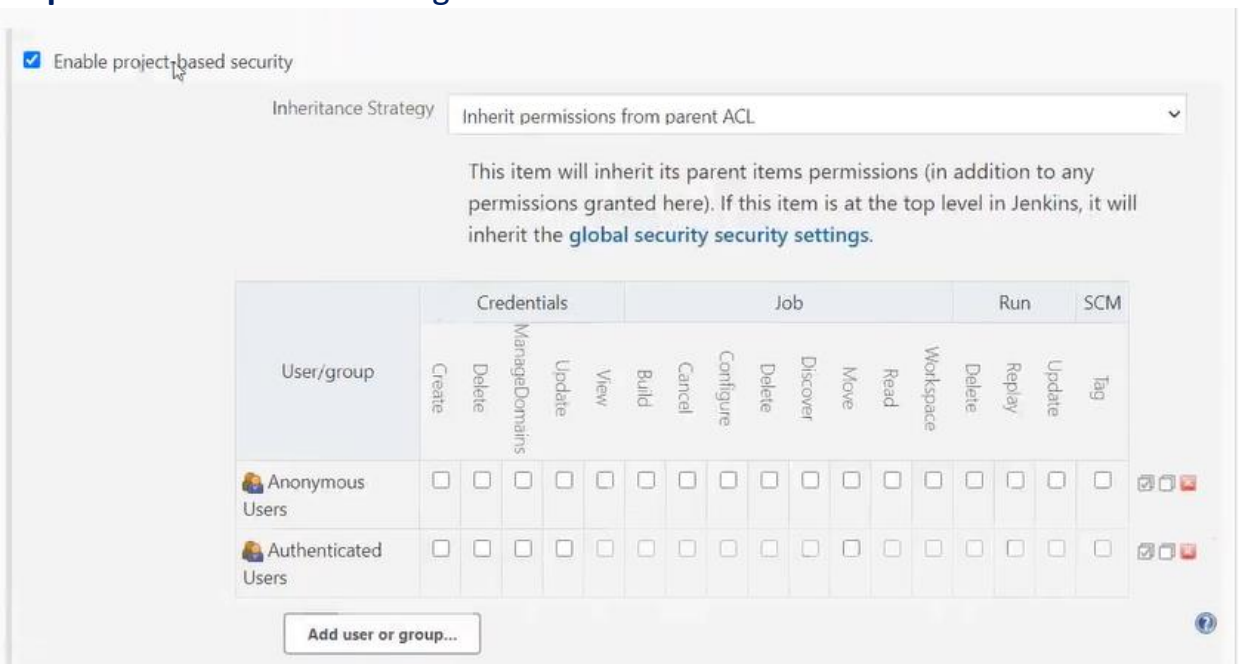
Project-based matrix authorization strategy Level Security?

step1: Login as an administrator **just enable project level security.** (Overall read permission)



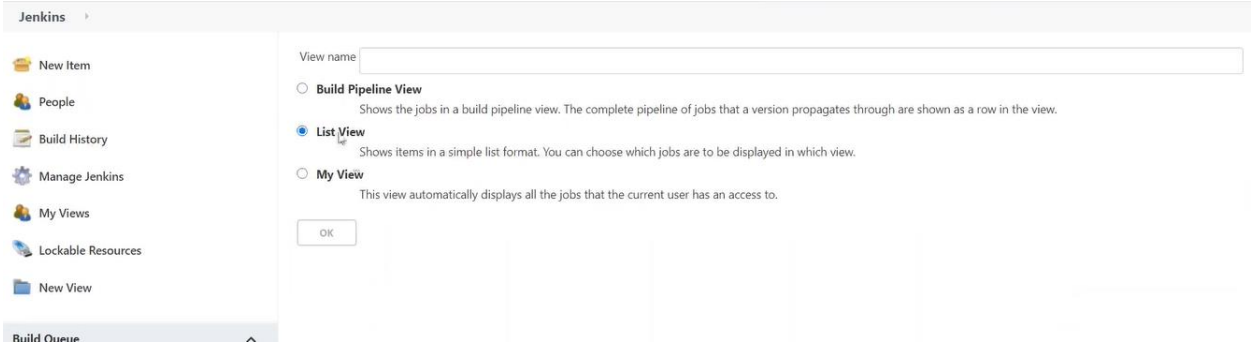
once you enabled you have to add the users

step2: Select the Job-> Configure.



step3: Click on the add user or group

grouping of jobs at view level.



Master Slave Architecture? Why we need? (Parallel running jobs)

UC: We have multiple jobs at same time.

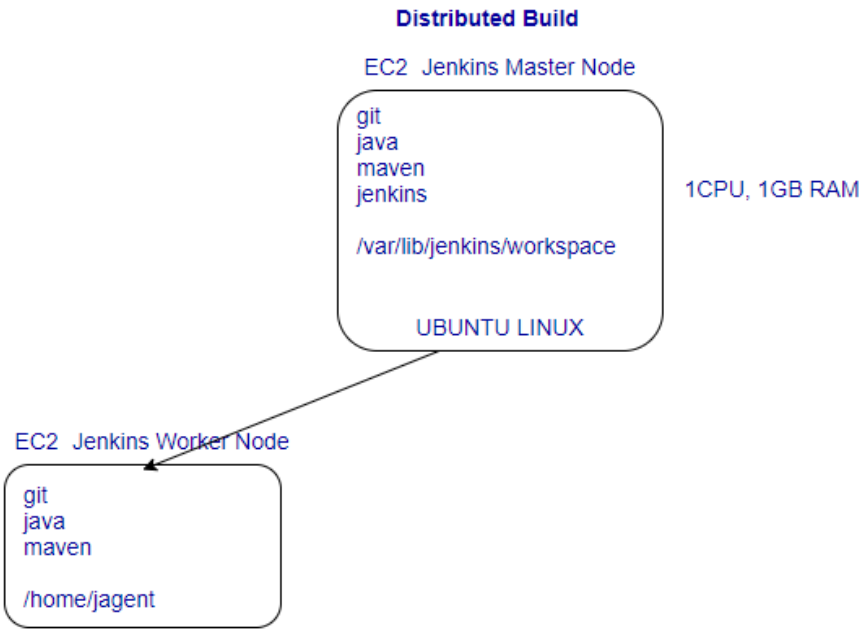
Recap: so far, we took a server (EC2 instance), install Jenkins on that server, running jobs on Jenkins's server, when we create a job it's created a workspace. Wherever you install the Jenkins is known as Jenkins's master (UI).

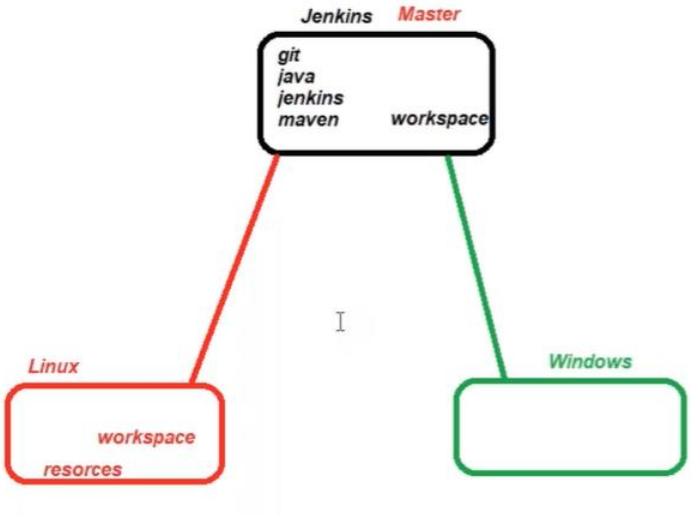
What is slave?

Slave always listen to the master, whenever master give instruction to the slave, slave executes those instructions. Once master is getting overloaded it needs some helping hands, so it extends its jobs and tasks to slave machines.e.g. If we install multiple applications on our mobile then mobile speed becomes slow, same happens with server also if load increases its performance getting slow down.

free -m

Once master is getting overloaded it delegates its workload to slave machines, if want to separate different types of tasks on different machine then also master delegates the task to slave based on the type of job.





Jenkins's master **pulls the code from remote GitHub repository every time there is code commit**. Jenkins master machine can delegate the jobs to agent server/machines, so that job will be run on agent sever, which means that we are utilizing the resources of agent machine, on request from Jenkins's master, slave carry out builds and tests and produce test reports. Every resource has their own capacity can't go beyond their capacity.

Objective is to distribute the load in multiple servers, no need to install jenkins on agent servers.

Master delegates the jobs to slave, master just only manage the jobs.

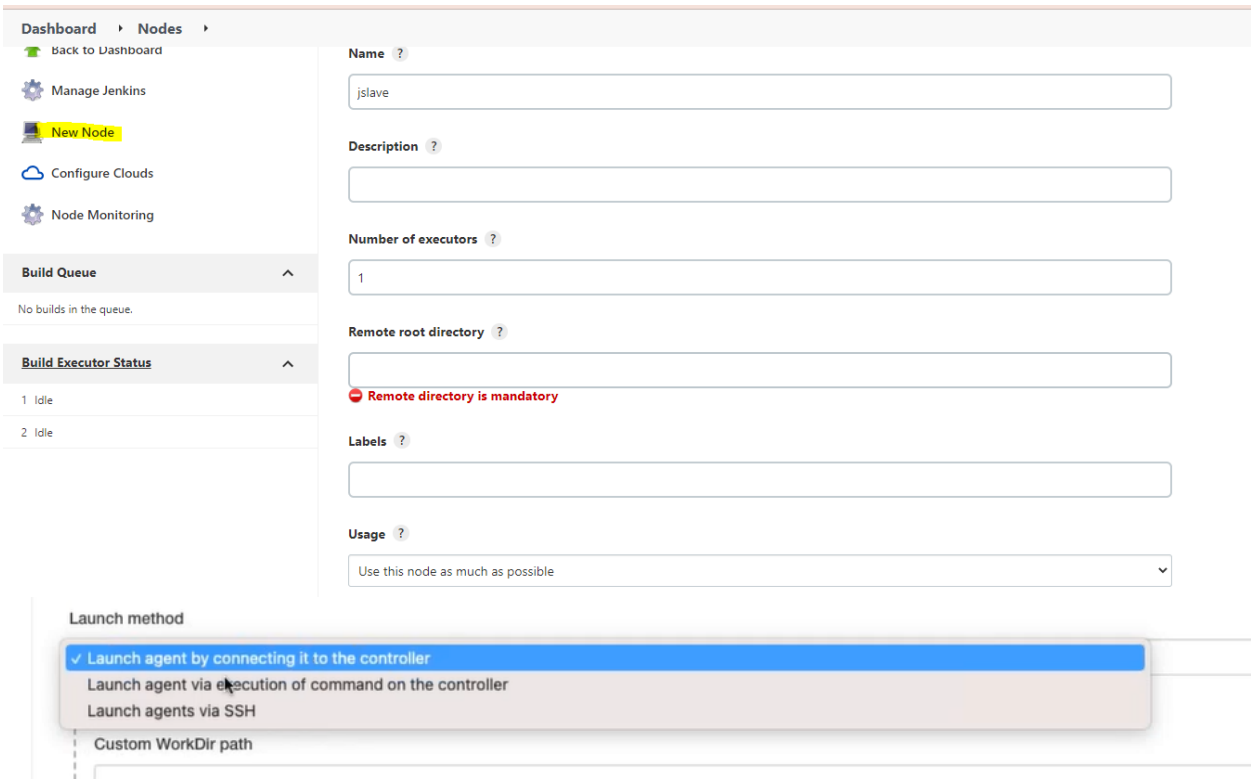
How to add slaves to the Jenkins master?

By default, master does not accept any connection from external resources.

There are different ways to establish a connection between master and slave.

Pre-requisite:

- Install java on all slaves.
- Install maven on all slaves (or use maven installed on Jenkins)



Way1: Using Agents

- Create a home directory on all slaves.

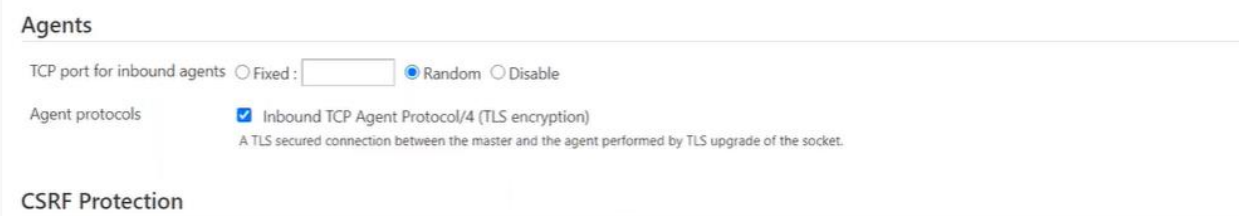
/*

- cd home
- mkdir jagent
- chmod 777 jagent/
- cd jagent

*/

```
root@jagent:~#
root@jagent:~#
root@jagent:~# cd /home/
root@jagent:/home# ls -l
total 8
drwxr-xr-x 5 lerndevops lerndevops 4096 Nov 16 03:02 lerndevops
drwxr-xr-x 3 ubuntu     ubuntu     4096 Nov 16 02:58 ubuntu
root@jagent:/home# mkdir jagent
root@jagent:/home# ls -l
total 12
drwxr-xr-x 2 root       root       4096 Nov 16 03:03 jagent
drwxr-xr-x 5 lerndevops lerndevops 4096 Nov 16 03:02 lerndevops
drwxr-xr-x 3 ubuntu     ubuntu     4096 Nov 16 02:58 ubuntu
root@jagent:/home# chmod 777 jagent/
root@jagent:/home# ls -l
total 12
drwxrwxrwx 2 root       root       4096 Nov 16 03:03 jagent
drwxr-xr-x 5 lerndevops lerndevops 4096 Nov 16 03:02 lerndevops
drwxr-xr-x 3 ubuntu     ubuntu     4096 Nov 16 02:58 ubuntu
root@jagent:/home# cd jagent/
root@jagent:/home/jagent#
root@jagent:/home/jagent#
root@jagent:/home/jagent# pwd
/home/jagent
root@jagent:/home/jagent#
```

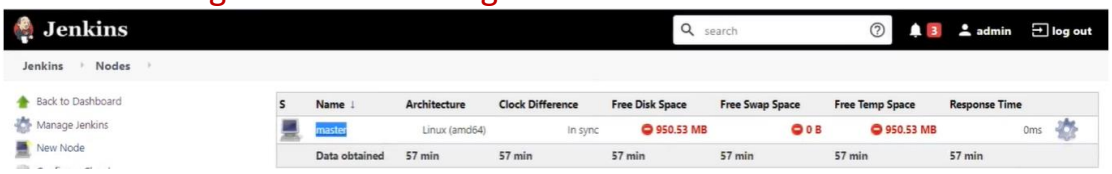
step1: Enable security (communication between two servers), Allow Agents.
Home->manage Jenkins-> Configure Global Security-> agents.



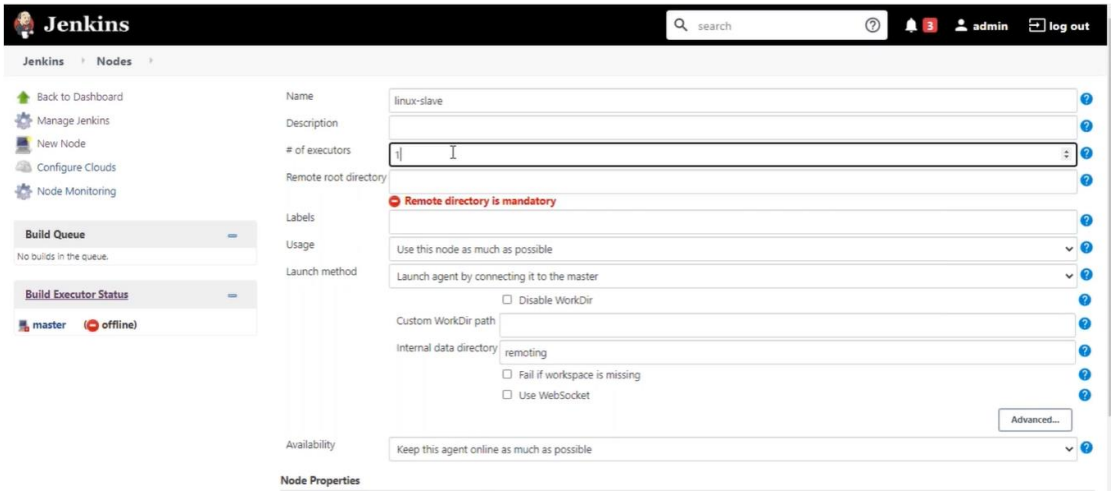
Change the setting to **Random** and **select** the Inbound TCP agent protocol/4 so that slave can communicate with the master.

step2: Add extra node (slave), tell Jenkins's master about slaves hey! Jenkins we are adding slaves for your help.

Home->manage Jenkins->Manage Nodes and Clouds



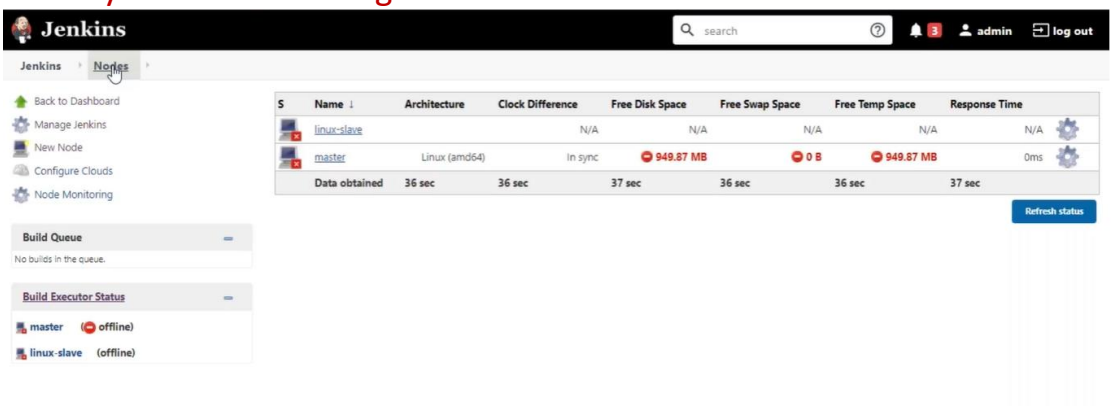
Click on new node



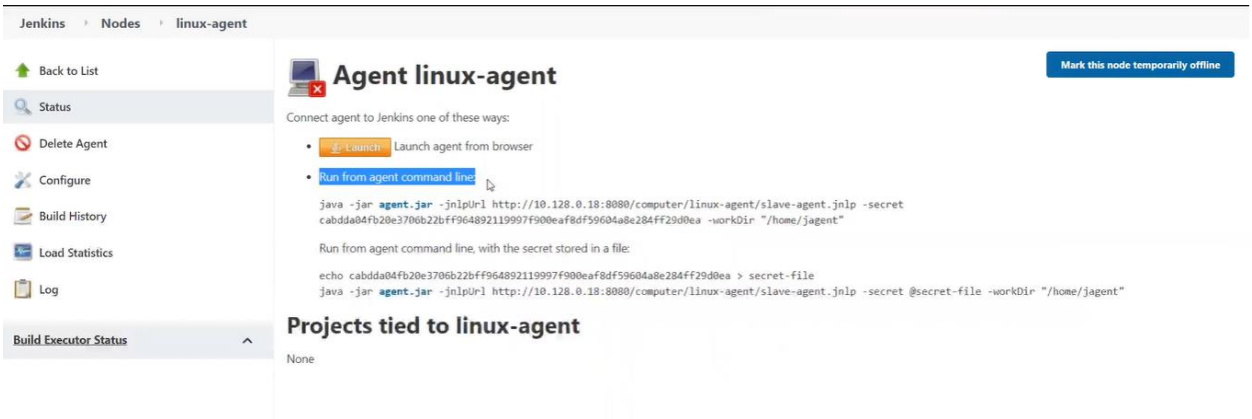
name: linux-slave
 # Of executers > The number of jobs that can be run in parallel on agent server
remote root directory: /home/jagent
where the workspace will be generated, home directory at slave just like a /var/lib/jenkins in master
labels: linux-slave
Usage: Use this node as much as possible (hey! master use this node first)
Launch method:

- **Launch agent by connecting it to the master.**
- Launch agent via execution of command on the master.
- Launch agents via ssh.

Availability: Keep this agent online as much as possible
When you save it linux-agent is still offline?



Click on linux-agent.



its looking for java and agent.jar in agent nodes.

1) Install the java
2) download the agent.jar
3) execute the command

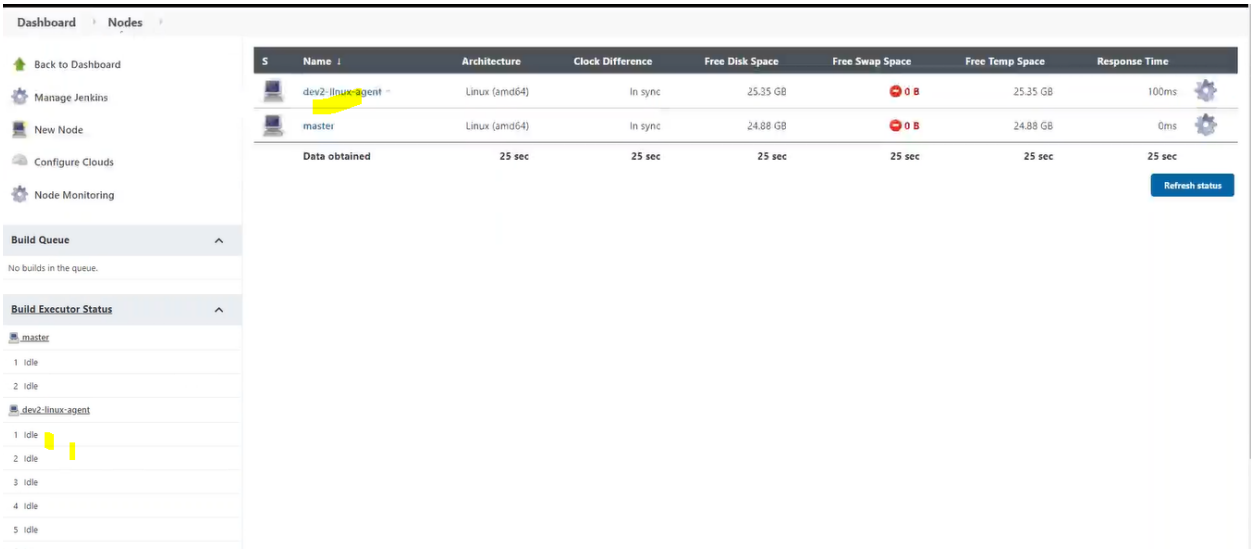
right click on agent.jar and save (copy link address)

step3: In slave node

cd /home/jagent

wget <copied url>

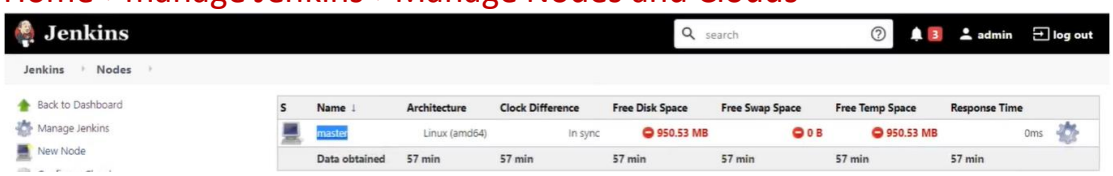
```
root@ip-172-31-29-111:/home/jslave#
root@ip-172-31-29-111:/home/jslave#
root@ip-172-31-29-111:/home/jslave# wget http://18.219.69.64:8080/jnlpJars/agent
.jar
--2020-07-12 17:39:16--  http://18.219.69.64:8080/jnlpJars/agent.jar
Connecting to 18.219.69.64:8080... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1522833 (1.5M) [application/java-archive]
Saving to: 'agent.jar'

agent.jar           100%[===================>]   1.45M  --.-KB/s    in 0.03s

2020-07-12 17:39:16 (52.3 MB/s) - 'agent.jar' saved [1522833/1522833]

root@ip-172-31-29-111:/home/jslave# ls -l
total 1488
-rw-r--r-- 1 root root 1522833 Jul 11 17:37 agent.jar
root@ip-172-31-29-111:/home/jslave#
```



step5: Click on job4-codecoverage
configure->general (**restrict where this job can be run**)

Way2:

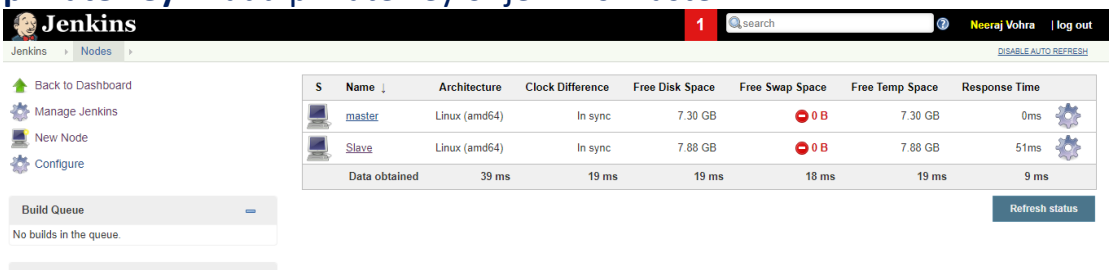
step1: setup password less authentication master has to access slaves
public / private **ssh key-pairs**
Our objective is master must access all the slaves, for which we have to setup the **ssh-authentication.**
In Linux every user has their own .ssh directory, in above example we have copied public key of Master's root user in slave1, when we try for ssh connection from master to slave by default Linux handshake of its public key (copied in slave1) with its private keys in to .ssh directory of the root user, if private key not available in .ssh folder then we have to give path of private key during ssh connection.
ssh **-i <path of private key directory>** root@slave1
In that case master can access slave but slave can't access master because we don't have copy slave's public key in master.
Keys are not specific to server its **specific to user (root) have private key** corresponding to shared public key.
Step2:
Home->manage Jenkins->Manage Nodes and Clouds



click on new nodes-> Permanent Agent
name: linux-slave
# of executers > The number of jobs that can be run in parallel on agent server
remote root directory: /home/jagent
where the workspace will be generated, home directory at slave just like a /var/lib/jenkins in master
labels: linux-slave grouping of slaves e.g., 3 servers are responsible for running frontend jobs and remaining 3 servers are responsible for running backend job(myslave)
Usage: Use this node as much as possible (hey! master use this node first)
Launch method:

- Launch agent by connecting it to the master.
- Launch agent via execution of command on the master.
- **Launch agents via ssh.**

Host->slave (will work because we have added the entry in /etc/hosts else enter ip)
Credentials-> username from **Add credential**
Host key verification strategy-> Non verification strategy
Save
Availability: Keep this agent online as much as possible
Add Credential
Home Page->Credentials->System->Global Credentials->add credentials
kind- ssh username with private key
Id - arbitrary name
Description->
username -> root // username login as
private key -> add private key of jenkins master



Delegate the jobs to Slave
Job->Configure-> **General**->Restrict Where this project can be run
In Slave before running specific job

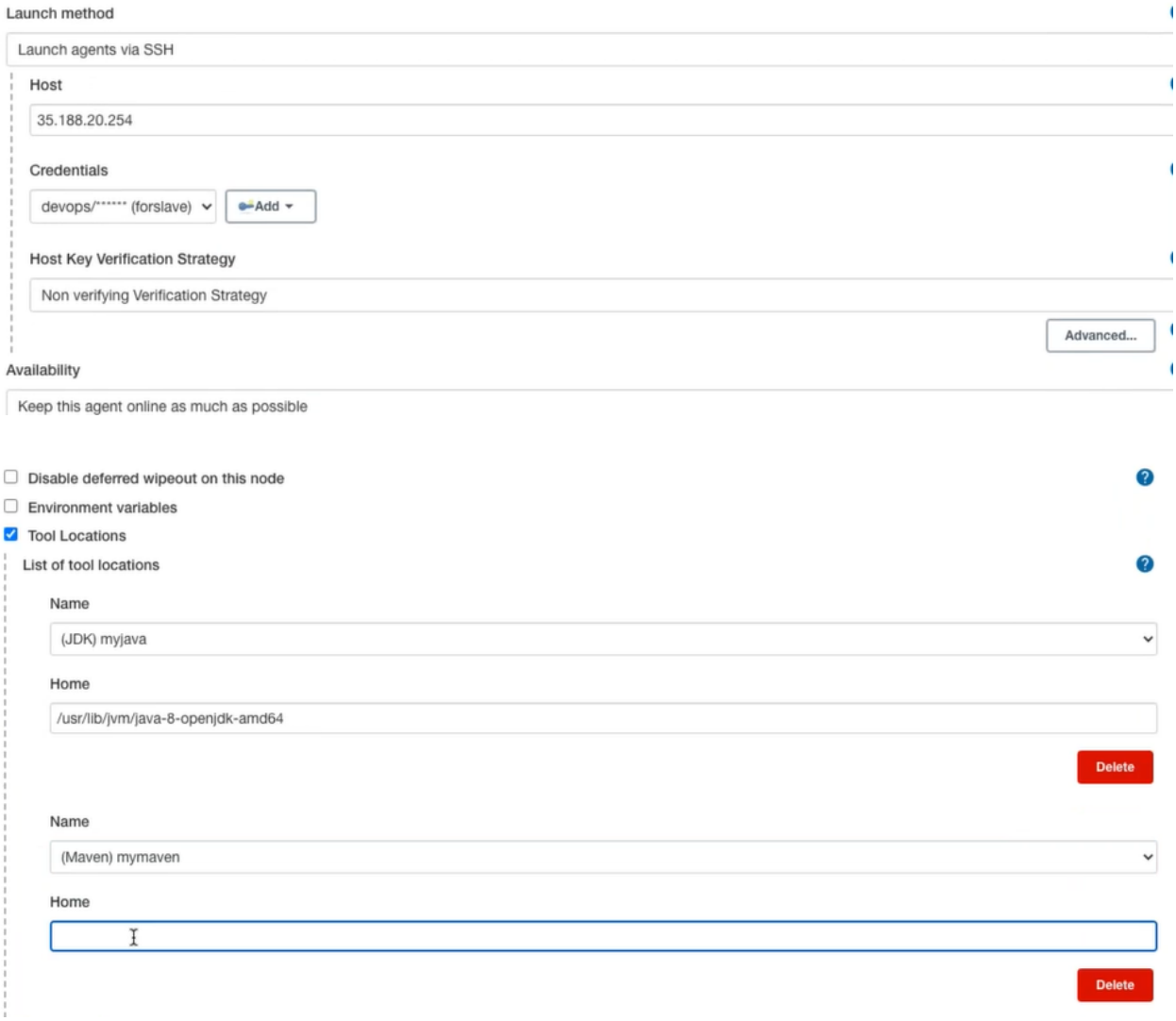
```
root@slave:/var/lib/jenkins# ls
remoting  remoting.jar
root@slave:/var/lib/jenkins#
```

In Slave after running specific job

Multiple Pipelines can be run from single master node, the output of the CI pipeline is, it gives you the artifacts.

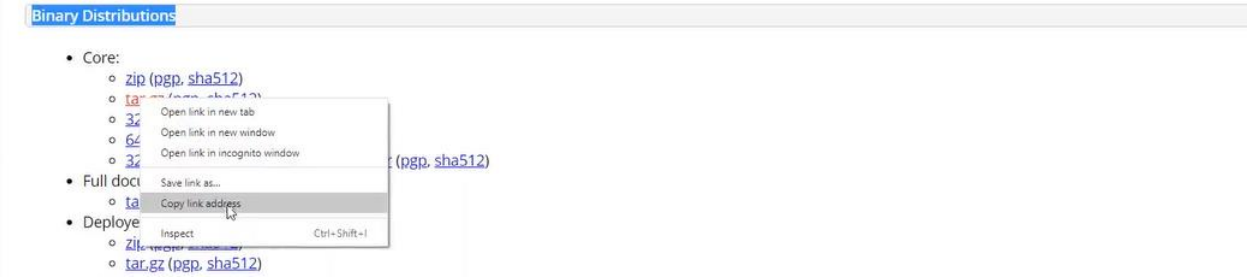
Way3:

cd /home/devops
mkdir jslave
chown devops:devops /home/devops/jslave



Example: tomcat

- download tomcat server



- wget <url>
- tar -xzf <name.gz> -C /opt // extract zip file
- cd /opt
- cd <name>/bin
- ./startup.sh
- ps -es | grep tomcat
- <public Ip>:8080
- cd webapp
- cd /home/jagent/workspace/package/target/sampleapp.war .
- ./stop.sh
- ./startup.sh
- <public Ip>:8080/sampleapp/